

TRANSCRIPT

Slide 1

Good morning, my name is Anne. This is my Unit 11 implementation of the research assistant our team designed in Unit 6. Two agents search and screen the literature for a health-sciences systematic review. A person decides what is kept, and only approved papers enter the results store. I will show how that design became a working system, then the live run, the reviewer's screen, the usability findings and the testing, including what the evidence cannot establish.

Slide 2

The design starts with a research question and its inclusion and exclusion criteria. The Planning Agent turns them into an ordered set of searches, then decides what to do after each result, while a shared record holds the question, the searches still to do and the results so far.

The design borrows from beliefs, desires and intentions, or BDI, as described by Rao and Georgeff in 1995. In Russell and Norvig's terms, it is a goal-based agent in a partly observed environment. It acts towards a stated goal, and it cannot see what a search failed to return, which is why every run reports its coverage rather than assuming it. Mapping it across: the research question is the goal, the planned searches are the intentions, and results update what the agent knows. There is no full BDI interpreter underneath. After each search, the planner uses what came back to choose its next

action, following the action-and-observation pattern in ReAct, by Yao and colleagues in 2023.

Searches go to PubMed and Semantic Scholar, with Crossref checking publication status. The Curation Agent then screens each paper, producing a relevance score, a reason, a summary, and an extract traceable back to the paper. The reviewer's decision is the only route into the results store, which is design constraint DC-08, and it never changed across the project.

Our proposal had considered an agent communication language, drawing on Finin and colleagues in 1994. A checked JSON record was enough here: it makes the handover explicit, and I can test it. What comes out is the list of approved papers.

I added both diagrams after the tutor's feedback on our Unit 6 proposal. The sequence diagram traces five error paths: a source not responding, a reply in the wrong shape, a plan the model cannot produce, the budget refusing a request, and the reviewer sending a paper back. The part of the class diagram on screen follows a single run. ResearchRequest holds the question and criteria, ResearchState carries the run's shared record, and RunBudget counts model requests and papers against their limits. Every record is strict, so a reply carrying an extra field or the wrong type is refused rather than guessed at.

Slide 3

So what stayed the same, and what changed? Both agents use the same configured Gemini model: the planner asks it for searches and for next actions, and the curator

asks it to screen papers against the criteria. LangGraph runs the steps, while ordinary Python checks the answers and enforces the limits. I kept deduplication, extract checking and storage in ordinary code because they need consistent rules, and putting the model in the way would add uncertainty and spend requests. The replaceable model adapter let me switch models through a single setting when the service began refusing requests.

I will focus on four changes during development and testing, each one recorded. First, out went the unused PubMed Central source. Screening and the extract checks work from titles and abstracts, and no application run ever called it. Leaving it in place would have implied full-text retrieval, which the system does not do.

Second, each run got a budget of 18 model requests, repairs and retries included. Every request is counted before it goes out. That limits the single run in front of you. It does not track the account's daily allowance or its token allowance.

The next changes came out of the live attempts on 17 and 18 September, where seven attempts failed to finish. Of those, six hit a service refusal, and one simply waited with no answer ever arriving. Google's 2026 troubleshooting guidance reads that refusal as the service being busy and says to wait and try again, which is what the run now does. I added a two-minute timeout on each connection operation, though a whole call can still run longer. Failures are retried after one, three and five minutes if the run has requests left, and the screen shows what is happening, with a Stop control.

The fourth change was the model itself. I pointed the live run at Gemini 3.5 Flash Lite through the existing adapter, and the eighth attempt completed. The run log records which model was used; Flash is still the default in the code. All four are documented against the original proposal.

Slide 4

Now to the agent actually working, on the live run of 18 September. I asked for original human studies of ivermectin as a COVID-19 treatment, with treatment outcomes, and reviews and prevention studies were excluded; the planner produced an ordered list of searches, each naming its purpose, its service and the search text. The log shows one completed PubMed search and one in Semantic Scholar, with 50 papers from each.

That response is typed; anything in the wrong shape is refused. After each search, the agent reads the result and picks between four actions: continue to the next search, refine this one with different words, drop it, or finish and hand over. A short reason comes with the choice and is kept with the planning decision. That is the loop the brief asks for: a goal arrives, the agent plans, it retrieves and processes, and it produces something a researcher can use.

On screen, the reviewer sets two numbers, both properly named on the form since the second usability session: the maximum number of search attempts, and the maximum results per search, which defaults to 50. That run spent 13 requests: 1 on the plan, 1 on the next-move decision after each of 2 searches, then 5 screening batches and 5 re-judgement batches.

There is also an offline demonstration in the submission, built from a single search and three invented papers, running on prepared replies, so it calls no service and needs no API key.

Slide 5

That brings me to retrieval and screening. Ordinary code looks for duplicates by identifier first, then by comparing titles, and it keeps the sources and searches that found each paper. Up to 50 papers are then given screening slots by taking turns across the searches, which is why the live run retrieved 100 and screened 50. The model did not pick the best 50. Anything left unscreened goes into the run's coverage report, along with any retrieval gap, and the reviewer sees that report at the top of the page. Keeping the searches and their limits follows PRISMA 2020 and PRISMA-S, described by Page and colleagues and Rethlefsen and colleagues in 2021. Those records alone do not make a review PRISMA-compliant.

The Curation Agent then asks Gemini to screen batches of ten papers, using their titles and abstracts with both sets of criteria. What comes back is agreement with those criteria as written, not relevance in a broader sense. For each paper, it must return include, exclude or uncertain, with a score, a one-sentence reason, a summary in the paper's own wording, and a supporting extract. Ten at a time means 50 papers take five requests when no retry or repair is needed, and that is what happened in the live run.

The next check hunts for that extract in the paper's title or abstract. If it is not there, approval is refused under that paper's own buttons, leaving the choice to flag it for a

new judgement or reject it. That is the red line in the image. What the check cannot do is tell us whether those words support the summary's meaning. That judgement stays with the reviewer.

The top image is the withdrawn ivermectin preprint by Elgazzar and colleagues, which the agent scored 10 out of 10 for relevance, and I flagged it. The extract on screen comes from an earlier version of the preprint, while Research Square's withdrawal notice sits on a later version. In the live run, one paper came back with no abstract at all, and the model used the message saying so as its own extract. The check rejected it, and so did I.

A guardrail runs through the whole process: the prompt puts the paper's text inside marked sections, and the tests check those boundaries hold. None of that rules out a paper whose own text influences the model's decision.

Slide 6

That is why the final decision stays with a person. I can approve a paper, flag it with a note, or reject it with a reason. To ask for another judgement, I press Send back, which offers the flagged papers alone or the flagged and rejected together. Choosing a decision sends nothing by itself. After revision, I decide again; only approvals enter the store, and undecided papers stay waiting.

To begin with, the review list only shows papers scoring above 5 out of 10, and the lower-scoring ones are still counted but hidden, unless they have already been decided or sent back. It keeps the list manageable, and it can also hide papers that are relevant.

The score itself is not calibrated. The model is asked for a number from 0 to 10; nothing defines what any point on that scale means, so the threshold is a display convenience rather than a measured cut-off. Approval protects what enters the store. It cannot recover a paper that was never put in front of me.

I checked the interface across two sessions, on 16 and 19 September, with one participant throughout: me, already familiar with the system. The first session covered desktop browsers, a phone, a range of widths and zoom levels, and both appearances. Out of it came clearer paper cards and a results screen that reopens an earlier run. Live use then turned up problems the session had not: the second judgement did not show clearly enough what had changed, so I made the changes visible.

In the follow-up session, the recorded task times were about 7 to 11 seconds. The first session had timed individual steps, so the two sets of figures are not directly comparable. My ease rating moved from 3 to 2, where 1 is easiest, but that rating was my own, and I did not run the System Usability Scale questionnaire. In 2022, Yu and colleagues measured a comparable screening tool on that scale, which is not something a single self-rating can stand in for. The session also found wording and feedback problems that led to further corrections, and it did not repeat the phone checks or the widths below 500 pixels. The testing document records the findings.

Slide 7

Now, what did the system achieve? The live run retrieved 100 papers, screened 50, and used 13 of the 18 model requests it was allowed. I sent 41 papers back for a second

judgement, and 4 of those recommendations changed. My final decisions came to 9 approvals and 41 rejections. Saving again created no duplicates, and the same nine identifiers appeared in the store and in the decision log.

The first evaluation used the CLEF TAR collection, published in 2019: 50 records from a single topic, carrying the collection's own relevance labels. I chose it because its PubMed identifiers and abstract-level labels are exactly what this tool works with. The sample holds all 15 relevant papers and 35 others, a richer share than the full topic has.

It measures four things, retrieval first: asking for 1,000 results per search, the agent retrieved 10 of the 15 relevant papers, a recall of 0.67. PubMed left one identifier out, and Semantic Scholar allows at most 100 results per request, which this implementation does not page through, so I record retrieval as partial.

Screening is a separate measurement, and I do not call it recall. Precision was 1.0, since every paper the agent marked as include was labelled include in the dataset, but positive coverage was only 0.40, which is 6 of the 15 relevant papers. The earlier Flash run reached 4. Coverage was higher here, though nothing in that makes one model generally better, and it still leaves 9 relevant papers unrecommended, which is why perfect precision on its own would not justify excluding papers automatically.

Traceability: the extract check accepted 41 of the 50 extracts, and I read 10 summaries against their sources, all 10 faithful. In the separate live sample, 17 of the 20 summaries were the extract word for word, so only 3 of them really tested whether the summary stays inside the paper.

Repeatability: screening the same records again shifted 1 decision out of 50, though 14 scores moved by 1 to 3 points, and the 6 include decisions held.

The second evaluation used a benchmark of my own: 100 records screened by hand, 8 of them relevant. Asking for 1,000 results per search, a run found all 8; the earlier live run, asking for 50, found none. Those were separate runs, so the larger request cannot be credited with the difference on its own, though it does expose the limit of the live run's setting. Both evaluations are small, and in my own benchmark I had already seen the labels. Method, result and limits for each measurement are in the testing document, so every figure here can be checked.

Slide 8

All of that rests on testing. The automated checks cover individual rules, the connections between them, and the complete screen workflow; on 1 October, 1,212 of them passed, with 3 skipped and 1 deselected, and Ruff came back clean. Functional tests run on invented replies prepared for the project, and live service checks are recorded separately. The export check refuses private recordings and looks for patterns such as keys and addresses, though recognising every secret is beyond what it can guarantee.

Testing turned up three examples worth showing here. Under R-21, a hostile check found an address sitting inside a structured model answer during reply approval. I corrected that path and added a test. Under R-24, one invalid approval could block an entire batch, so I moved the refusal beside the paper and kept the storage check, with

app and workflow tests covering it. Under R-25, a model request simply kept waiting. That is what produced the connection timeouts and the longer waits between retries, now covered by tests.

The code comment on screen explains why storage checks the extract a second time. A record can change after the first check, so one unit test replaces the extract after validation and confirms that storage refuses it, putting the rule where the data is saved. Raw outputs sit under evidence, so every one of these traces back to the run that produced it.

Slide 9

None of this arrived finished, and the timeline marks the turning points. The project began on 26 August, and that first day was spent discovering that the antivirus was blocking every API call. Corrections for privacy and publication rights followed on 2 September, and the tutor's feedback on our Unit 6 proposal, on 14 September, is what brought in the two diagrams and the standard-dataset tests. I ran the first usability session on 16 September. Over the next two days, I rebuilt the screen from my own mock-ups, against seven attempts that would not finish. The eighth completed on 18 September, on a different model, and the dataset repeat ran that same evening. On 19 September I closed the final measurements and the second usability session ran the same day. The code baseline is 20 September, and review and correction carried on well past that date, ending with the public copy checked from a fresh clone in October.

Slide 10

To close, the system takes a research question through searches and screening to a stored list of approved papers. Decisions and their supporting extracts can be checked afterwards. Its main weakness is coverage: relevant papers can be missed or hidden before the reviewer ever sees them.

The evidence amounts to one completed live workflow, 50 CLEF records and the 100 labelled during development. I tested the interface myself, so neither performance on new topics nor usability for new reviewers is established. I cannot claim that the tool makes research faster or more accurate.

The completed run used Flash Lite, and service availability and account limits can both change.

My next steps would be a new topic with labels I had not seen during development, then testing with new reviewers.

Data comes from PubMed, with the NCBI disclaimer shown on screen, and from Semantic Scholar, Crossref, and the CLEF TAR collection used for the standard dataset. The main libraries are LangGraph, Pydantic, Streamlit and SQLite, together with the Gemini SDK, the model cards published by Google DeepMind in 2026 for Flash Lite and Flash, pytest, and Ruff. Thank you. The repository, the instructions for running the system, and the offline demonstration are all in the submission.

AI Acknowledgement: Generative AI tools (Grammarly and Claude) were used for proofreading, language refinement, and formatting support. All analyses, interpretations, conclusions, recommendations, slide content, and presentation structure are my own work.

References

CLEF-TAR (2019) *Technologically Assisted Reviews in Empirical Medicine*. Available at: <https://github.com/CLEF-TAR/tar> (Accessed: 14 September 2026).

Elgazzar, A. et al. (2020) *Efficacy and Safety of Ivermectin for Treatment and prophylaxis of COVID-19 Pandemic* [Withdrawn preprint, version 1]. Research Square. Available at: <https://doi.org/10.21203/rs.3.rs-100956/v1>.

Finin, T., Fritzson, R., McKay, D. and McEntire, R. (1994) 'KQML as an agent communication language', *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)*. Gaithersburg, MD, 29 November–2 December. New York: ACM, pp. 456–463. Available at: <https://doi.org/10.1145/191246.191322>.

Google (2026) *Troubleshooting guide*. Available at: <https://ai.google.dev/gemini-api/docs/troubleshooting> (Accessed: 14 September 2026).

Google DeepMind (2026a) *Gemini 3.5 Flash-Lite: model card*. Available at: <https://deepmind.google/models/model-cards/gemini-3-5-flash-lite/> (Accessed: 19 September 2026).

Google DeepMind (2026b) *Gemini 3.6 Flash: model card*. Available at: <https://deepmind.google/models/model-cards/gemini-3-6-flash/> (Accessed: 27 August 2026).

Page, M.J. et al. (2021) 'The PRISMA 2020 statement: an updated guideline for reporting systematic reviews', *International Journal of Surgery*, 88, 105906. Available at: <https://doi.org/10.1016/j.ijsu.2021.105906>.

Rao, A.S. and Georgeff, M.P. (1995) 'BDI agents: from theory to practice', *Proceedings of the First International Conference on Multiagent Systems (ICMAS-95)*. San Francisco, CA, 12–14 June. Menlo Park, CA: AAAI Press, pp. 312–319.

Research Square (2021) *WITHDRAWN: Efficacy and Safety of Ivermectin for Treatment and prophylaxis of COVID-19 Pandemic* [Withdrawal notice, version 4; editorial note updated 12 March 2022]. Available at: <https://doi.org/10.21203/rs.3.rs-100956/v4>.

Rethlefsen, M.L. et al. (2021) 'PRISMA-S: an extension to the PRISMA statement for reporting literature searches in systematic reviews', *Journal of the Medical Library Association*, 109(2), pp. 174–200. Available at: <https://doi.org/10.5195/jmla.2021.962>.

Russell, S.J. and Norvig, P. (2022) *Artificial intelligence: a modern approach*. 4th edn, Global Edition. Harlow: Pearson Education Limited.

UNESCO (2022) *Recommendation on the ethics of artificial intelligence*. Paris: UNESCO. Available at: <https://unesdoc.unesco.org/ark:/48223/pf0000381137> (Accessed: 30 September 2026).

Yao, S. et al. (2023) 'ReAct: synergizing reasoning and acting in language models', *Eleventh International Conference on Learning Representations (ICLR 2023)*. Kigali, Rwanda, 1–5 May. Available at: <https://doi.org/10.48550/arXiv.2210.03629>.

Yu, F., Liu, C. and Sharmin, S. (2022) 'Performance, usability, and user experience of Rayyan for systematic reviews', *Proceedings of the Association for Information Science and Technology*, 59(1), pp. 843–844. Available at: <https://doi.org/10.1002/pra2.745>.